

String in Java

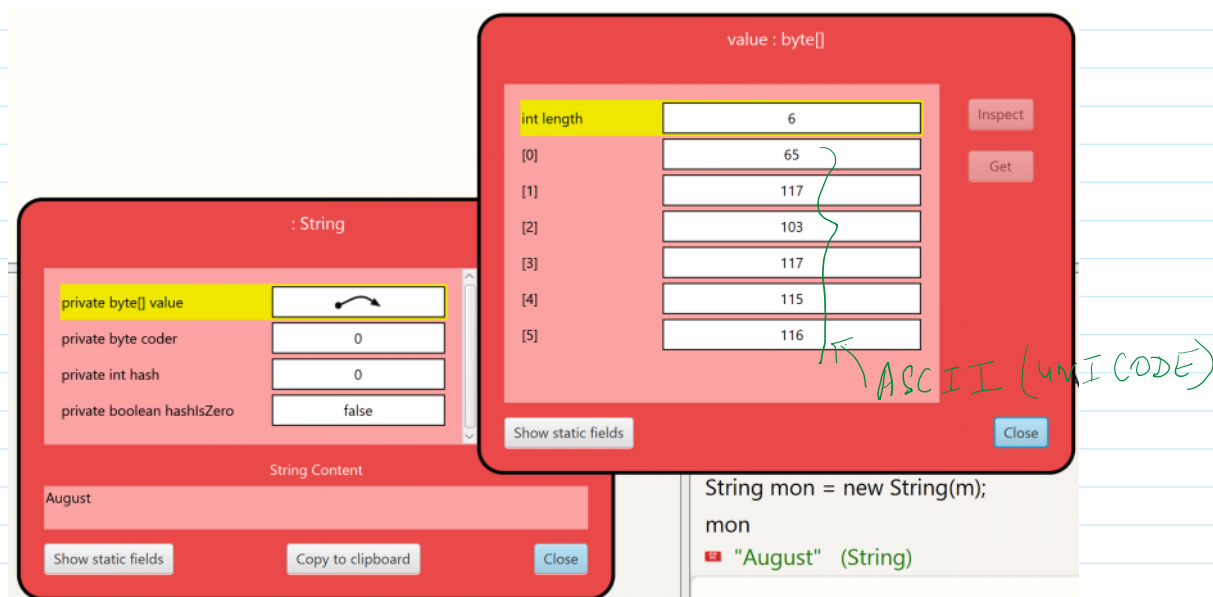
13 August 2026 05:19

In Java, String is basically an object that represents sequence of char values. An array of characters works as a string in Java. For example

```
char[] m = {'A', 'u', 'g', 'u', 's', 't'};
m
<object reference> (char[])
System.out.print("\nm = " + m);
System.out.print("\nm = " + java.util.Arrays.toString(m));
m
<object reference> (char[])
String mon = new String(m);
mon
"August" (String)
```

unsigned
↓
1 byte → 0 to 255
-128 to 127

A u g u s t
byte[] = 65



Java String class provides a lot of methods to perform operations on strengths such as compare(), concat(), equals(), split(), length(), replace(), compareTo(), intern(), substring() etc.

What is String in Java?

Generally, a string is a sequence of characters. However, a String in Java is an object that represents a sequence of characters and the **java.lang.String** class is used to create String object. Java strings are immutable. That means their values cannot be changed once created.

There are two ways to create a String object:

1. By String literal
2. By new Keyword

Creating String Using String Literal

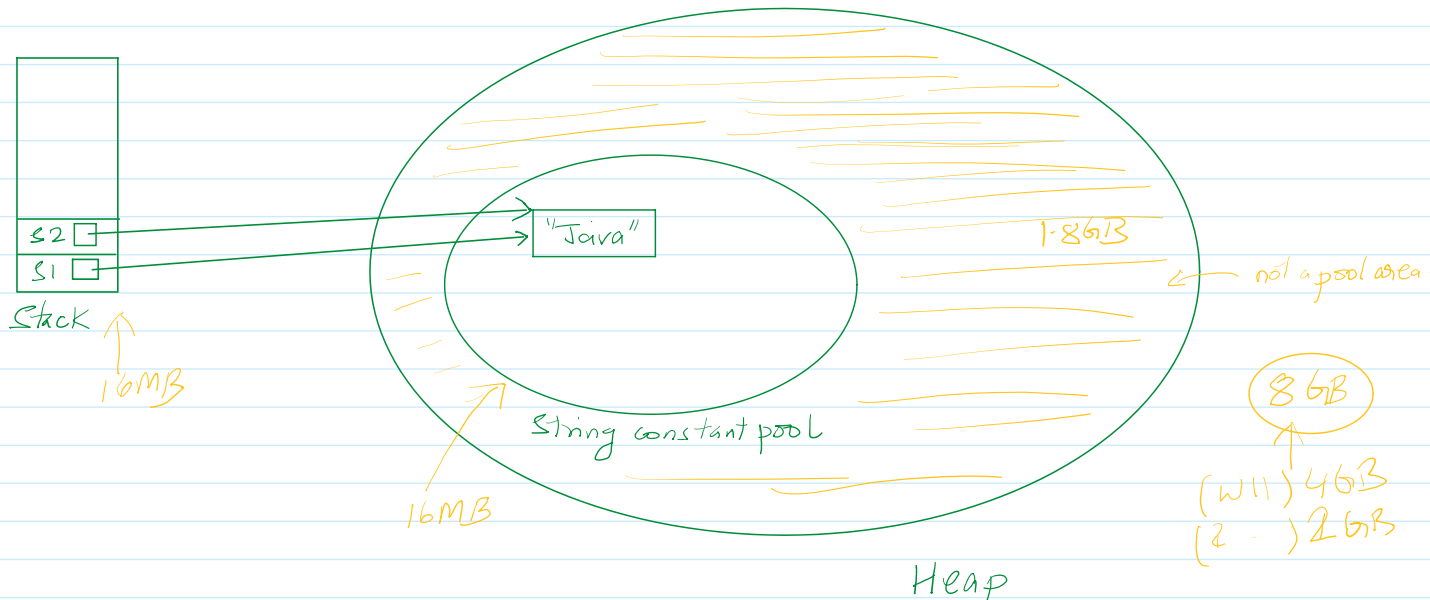
Java string literal is created by using double quotes. For example:

```
String str = "Welcome to Programming in Strings";
```

```
str
"Welcome to Programming in Strings" (String)
```

Each time you create a string literal. The JVM checks the string constant pool first. If the string already exists in the pool, a reference to the pooled instance is returned. If the string doesn't exist in the pool, a new String instance is created and placed in the pool. For example

```
String s1 = "Java";
String s2 = "Java"; // It doesn't create a new instance.
```



In the above example, only one object will be created. Firstly, JVM will not find any string object with the value "Java" in String constant pool that is why it will create a new object. After that it will find the string with the value "Java" in the pool. It will not create a new object but will return the reference to the same.

```
package MyStrings;
import java.util.*;

/**
 * Write a description of class MyStringOne here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MyStringOne
{
    public static Scanner sc = new Scanner(System.in);
    public static void main(String[] args){
        String s1 = "Welcome";
        String s2 = "Welcome"; //instance doesn't create
        System.out.print("\n"+s1+" " + s2);
    }
}
```

```
}
```

Output:

Welcome Welcome

Creating String Using new Keyword

A string can be created using the new keyword, which explicitly creates a new String object in memory rather than using the string literal pool. For example:

```
String s = new String("Welcome");// Creates 2 objects and 1 reference variable
```

Here,

"Welcome" is a string literal, so it is stored in the string pool(one object).

"new String("Welcome") creates a new String object in the heap(second object).

's' is the reference variable that points to the object in the heap.

In such case JVM will create a new String object in normal (non-pool) Heap memory and the literal welcome. Will be placed in the string constant. The variable "s" will refer to the object in the heap (non - pool).

compareTo()

Method returns:

```
int v = str1.compareTo(str2);
```

If the value of v is positive then str1 comes after str2.

If the value of v is negative then str1 comes before str2.

If the value of v 0 then both the strings are equal.

```
if (str1 == str2){ //it compares the reference value
```

```
}
```

```
String s1 = new String("Java");
```

```
String s2 = new String("Java");
```

```
s1
```

```
"Java" (String)
```

```
s2
```

```
"Java" (String)
```

```
s1 == s2
```

```
false (boolean) ← reference is not same
```

```
String s3 = "Java";
```

```
s1 == s3
```

```
false (boolean) ←
```

```
String s4 = "Java";
```

```
s3
```

```
"Java" (String)
```

```
s4  
"Java" (String)  
s3 == s4  
true (boolean)
```

← Reference is same because s3 & s4 belongs to
String constant (literal) pool.

Therefore we have to compare the content of the string using equals() or compareTo() methods.

"" => empty string or blank string